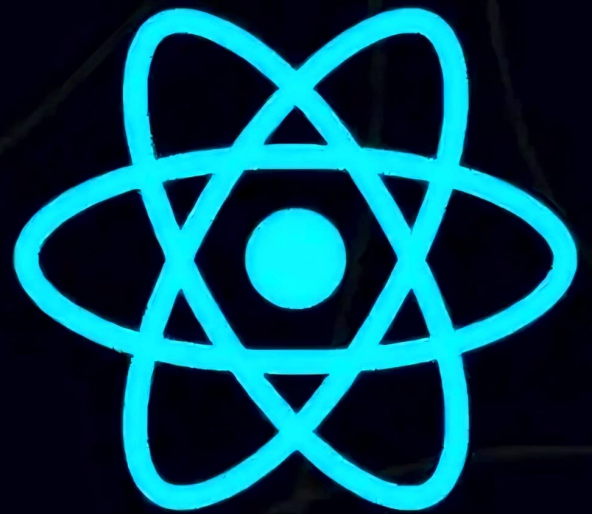


List & Keys in React



List

In `React`, a list is a way to display a `collection of data`, often rendered as a series of elements. The most common way to create a list is to use the `map()` function to iterate over an array and return a `JSX element` for each item.



Example

Imagine you have a list of fruits that you want to display

```
const fruits = ['Apple', 'Banana', 'Cherry'];

function FruitList() {
  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  );
}

export default FruitList;
```

Output

- Apple
- Banana
- Cherry

Keys

Keys are a crucial aspect when rendering lists in React. They help React **identify** which items have changed, are added, or are removed. This **improves performance** and helps React efficiently update and **re-render** components.

A key should be a **unique** and **stable** identifier for each item in the list. It allows React to keep **track of elements** between renders.

Example

Using the previous example, let's add unique keys to each list item

```
const fruits = [
  { id: 1, name: 'Apple' },
  { id: 2, name: 'Banana' },
  { id: 3, name: 'Cherry' },
];

function FruitList() {
  return (
    <ul>
      {fruits.map(fruit => (
        <li key={fruit.id}>{fruit.name}</li>
      ))}
    </ul>
  );
}

export default FruitList;
```

In this updated example, **Unique Keys**: Each fruit object now has an id property (id: 1, id: 2, id: 3).

Using Keys: The key attribute in the element is set to **fruit.id**. This provides React with a **unique identifier** for each list item.

Why Keys Are Important ?

Efficiency: Helps React minimize re-renders by only updating the changed elements.

Stability: Ensures the correct item is referenced even if the order of items changes.

By understanding and using **lists** and **keys** correctly, you can create more efficient and reliable React applications.

